第四章作业

1 最小覆盖子串

给定字符串 `S`（长度 n）和 `T`（长度 m），请返回 `S` 中能覆盖 `T` 所有字符（每种字符出现次数不少于 `T` 中对应次数）的最短子串及其起始下标（从 0 开始）；若不存在，返回 `("", -1)`。若 `T` 为空，返回 `("", 0)`。

例如 `S = "ADOBECODEBANC"`，`T = "ABC"`，返回 `("BANC", 9)`。

要求：时间复杂度 $O(n+m)$（或 $O(n)$，系数合理即可），可借助 $O(|\Sigma|)$ 额外空间。请说明思路并给出伪代码。

> 原题：力扣 76. 最小覆盖子串（力扣只要求返回子串，本题还要求返回起始下标）。
>
> **T76.最小覆盖子串**
>
> https://leetcode.cn/problems/minimum-window-substring/

解： 思路：由问题的单调性，可以使用滑动窗口方法

```
class Solution {
public:
    int cnt[255];
    pair<string, int> minWindow(string s, string t) {
        if (t == "") return ("", 0);
        int debt = 0;
        for (char c: t) {
            cnt[c]--;
            debt++;
        }
        int len = 1e6;
        int l = -1;
        int start = -1;
        for (int r = 0; r < s.size(); r++) {
            int c = s[r];
            if (cnt[c] < 0) {
                debt--;
            }
            cnt[c]++;
            if (debt == 0) {
                while(true) {
                    l++;
                    c = s[l];
                    cnt[c]--;
                    if (cnt[c] < 0) {
                        l--;
                        cnt[c]++;
                        break;
                    }
                }
                if (len > r - l) {
```

```
                    start = l + 1;
                    len = r - l;
                }
            }
        }
        return (start == -1 ? "" : s.substr(start, len), start);
    }
};
```

---

2 KMP 求所有出现位置（允许重叠）

给定模式串 `P`（长度 m）和文本串 `T`（长度 n），`P` 与 `T` 均只含普通字符（无通配符）。请计算并返回 `P` 在 `T` 中**所有**出现的位置的起始下标（从 0 开始，**允许重叠**）。若 `P` 为空，返回空列表。

例如 `P = "abab"`，`T = "abababab"`，返回 `[0, 2]`。

要求：时间复杂度 $O(n+m)$，除结果列表外的额外空间 $O(m)$（用于课件定义的 `next` 数组）。请说明思路并给出伪代码。

> 改编自：力扣 28. 找出字符串中第一个匹配项的下标（力扣只求第一次出现，本题求全部出现且允许重叠）。
>
> **E28.找出字符串中第一个匹配项的下标**
>
> https://leetcode.cn/problems/find-the-index-of-the-first-occurrence-in-a-string/

思路：每次成功匹配后，j = next[m]

```
KMPAll(P, T):
    m ← |P|
    if m == 0: return []
    next ← BuildNext(P)
    ans ← []
    j ← 0                    // 当前已匹配的 P 前缀长度

    for i ← 0 to |T| - 1:
        while j > 0 and T[i] != P[j]:
            j ← next[j]

        if T[i] == P[j]:
            j ← j + 1

        if j == m:
            ans.append(i - m + 1)
            j ← next[m]      // 继续匹配，保留可能重叠的后缀

    return ans
```

---

3 利用 next 数组求最短周期

给定字符串 S（长度 n），请利用 KMP next 数组计算 S 的**最短周期**。若 S 为空，返回 0。

若 S 可由某个长度 $p < n$ 的子串重复多次构成（即 $S = X^k$ 且 $|X| = p$），则称 p 为 S 的一个周期，否则记最短周期为 n（即 S 本身）。

例如：

- S = "abcabcabc" → 最短周期 3（"abc" 重复 3 次）
- S = "abcab" → 最短周期 5（无非平凡周期）
- S = "aaaa" → 最短周期 1

要求：时间复杂度 $O(n)$，额外空间 $O(n)$（用于 next 数组）。请说明思路并给出伪代码。

> 对应题：力扣 459. 重复的子字符串（判断是否存在非平凡周期，本题进一步求最短周期）；OpenJudge 02406 字符串乘方（求 $S = a^k$ 的最大 k，即 n/p）。
>
> **E459.重复的子字符串**
>
> https://leetcode.cn/problems/repeated-substring-pattern/
>
> **02406: 字符串乘方**
>
> http://cs101.openjudge.cn/practice/02406/

思路：求最小循环节长度n - next[n]即可，注意判断是否整除

```
ShortestPeriod(S):
    n ← |S|
    if n == 0: return 0

    next ← BuildNext(S)
    p ← n - next[n]

    if n mod p == 0:
        return p
    else:
        return n
```

---

4 单次遍历原地删除 'b' 与级联 "ac"

给定字符串 S（长度 n，以字符数组形式存储，可就地修改）。

在一次从左到右扫描中，**原地**（$O(1)$ 额外空间）删除 S 中所有字符 'b'，并删除所有出现的子串 "ac"（删除要**级联**，即删除某个 "ac" 可能使得原本不相邻的 a 与 c 变为相邻，这些新形成的 "ac" 也必须被删除）。要求**单次遍历**（只能从左向右扫一次），最终把保留的字符移到 S 的左端，并返回新长度。

例如：

- "acbac" → ""（全部被删）

- `"ababac"` → `"aa"`
- `"aaccac"` → `""`（级联删除）

请给出：**思路说明、伪代码、以及时间复杂度分析**。

> 非力扣原题，是经典面试题（Techie Delight 上有同构题 "Remove all occurrences of AB and C"）。
>
> **E2696.删除子串后的字符串最小长度**（删除 `"AB"`、`"CD"`，级联）
>
> https://leetcode.cn/problems/minimum-string-length-after-removing-substrings/
>
> **E1047.删除字符串中的所有相邻重复项**
>
> https://leetcode.cn/problems/remove-all-adjacent-duplicates-in-string/

思路：利用栈储存

```
RemoveBAndACInPlace(S):
    write ← 0

    for read ← 0 to |S| - 1:
        c ← S[read]          // 先读取，避免覆盖待处理字符

        if c == 'b':
            continue

        if c == 'c' and write > 0 and S[write - 1] == 'a':
            write ← write - 1
        else:
            S[write] ← c
            write ← write + 1

    return write             // 最终内容为 S[0:write]
```

时间复杂度：每个字符最多出入栈一次，从而时间复杂度为$O(n)$